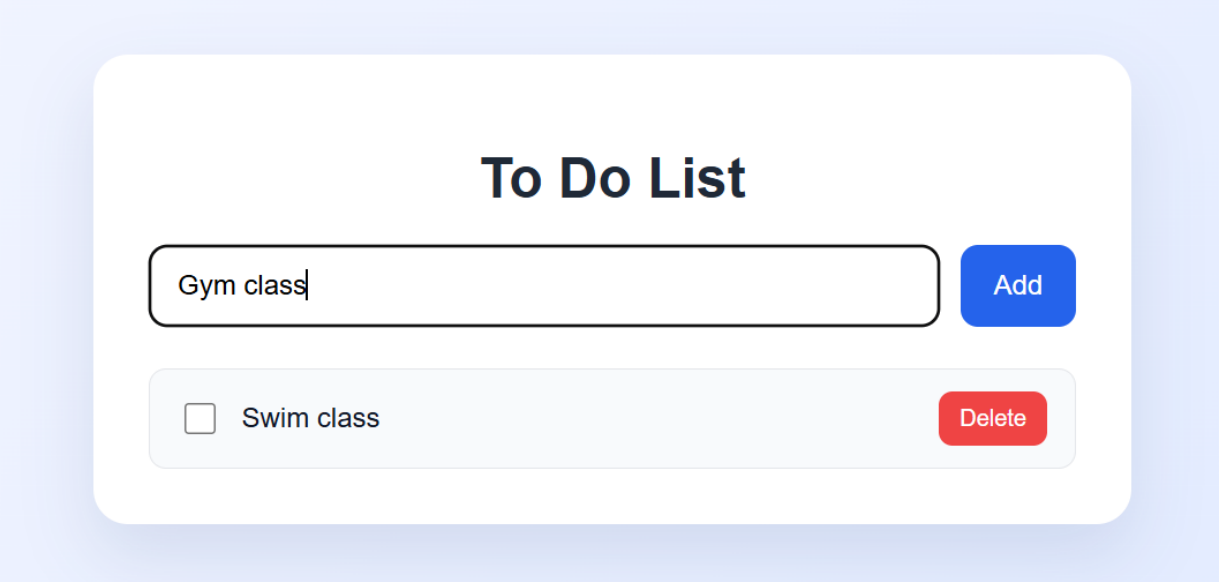
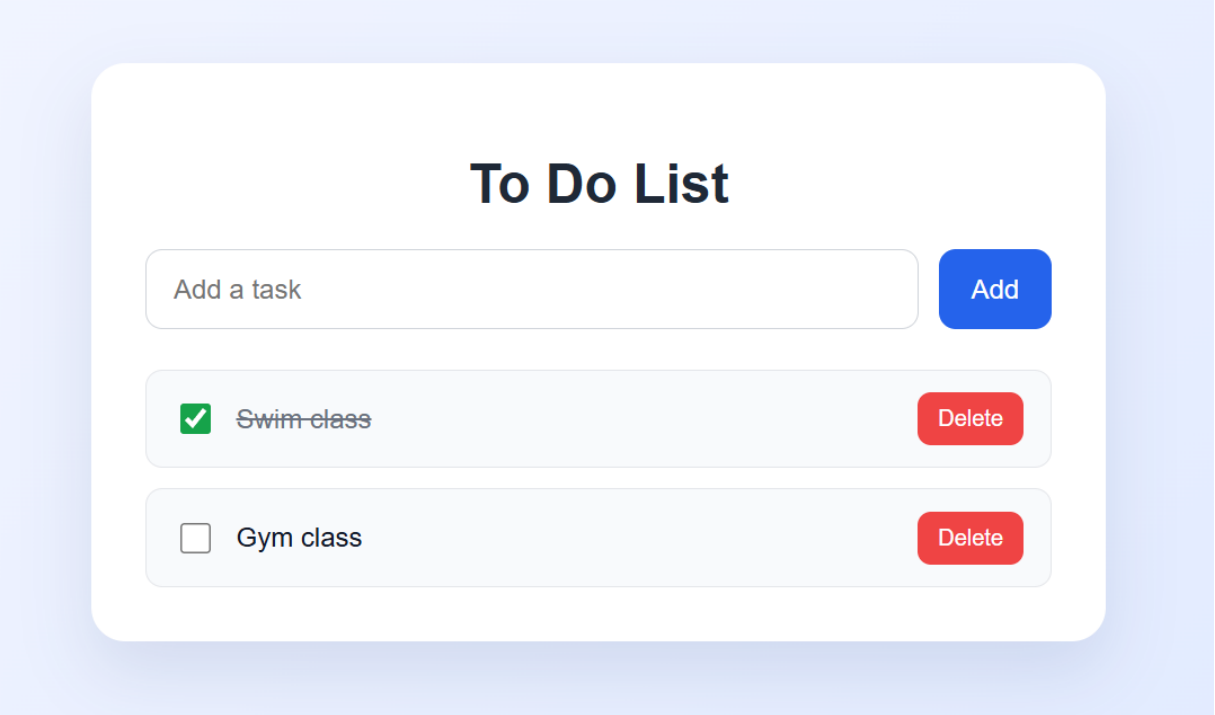
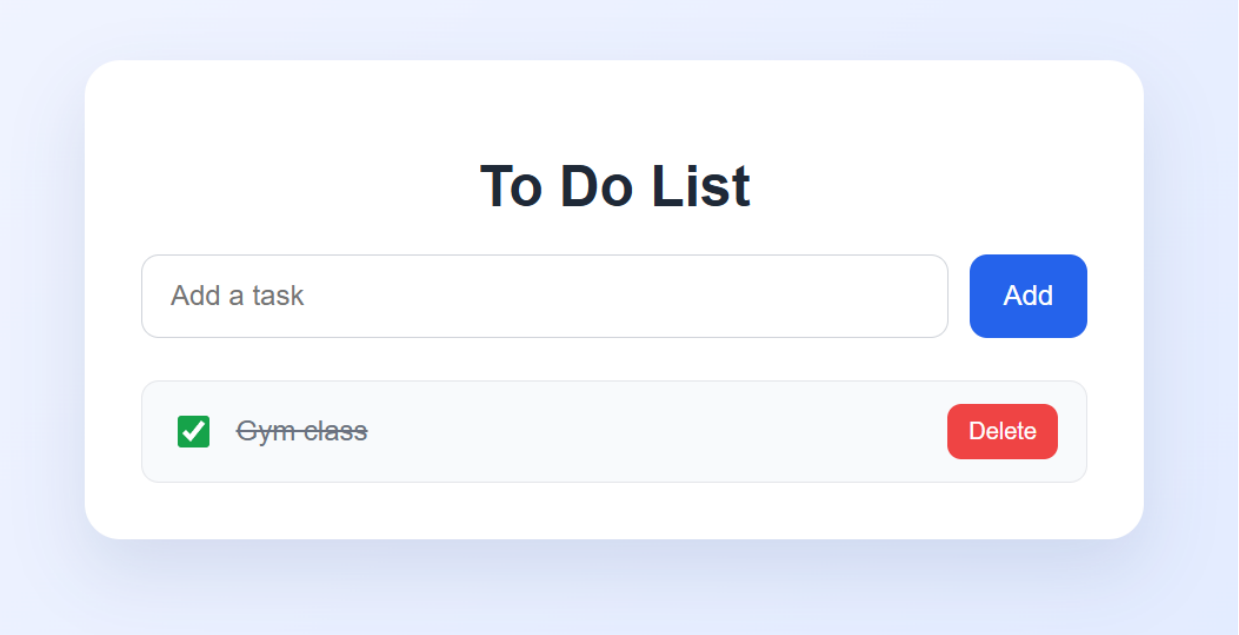
Q1.

The app lets users add tasks, mark them complete, and delete them. Tasks are stored in the browser's `localStorage`, so they remain available after the page is refreshed.








```
function loadTodos() {
  try {
    const stored = localStorage.getItem(STORAGE_KEY);
    return stored ? JSON.parse(stored) : [];
  } catch (error) {
    return [];
  }
}

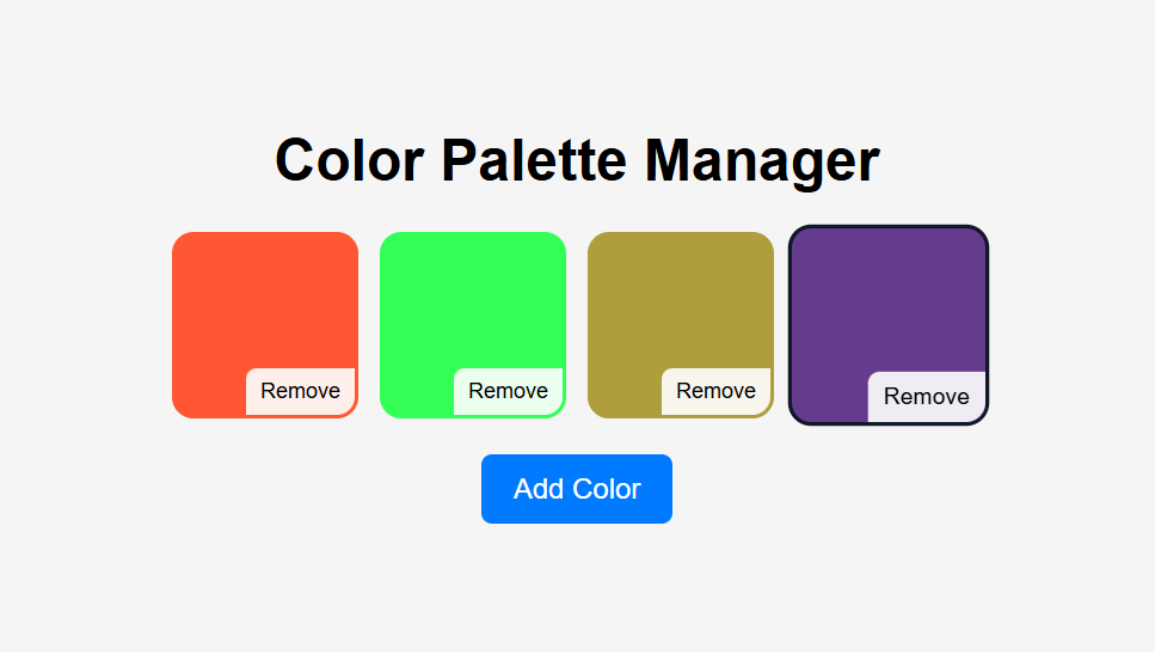
function saveTodos(items) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
}

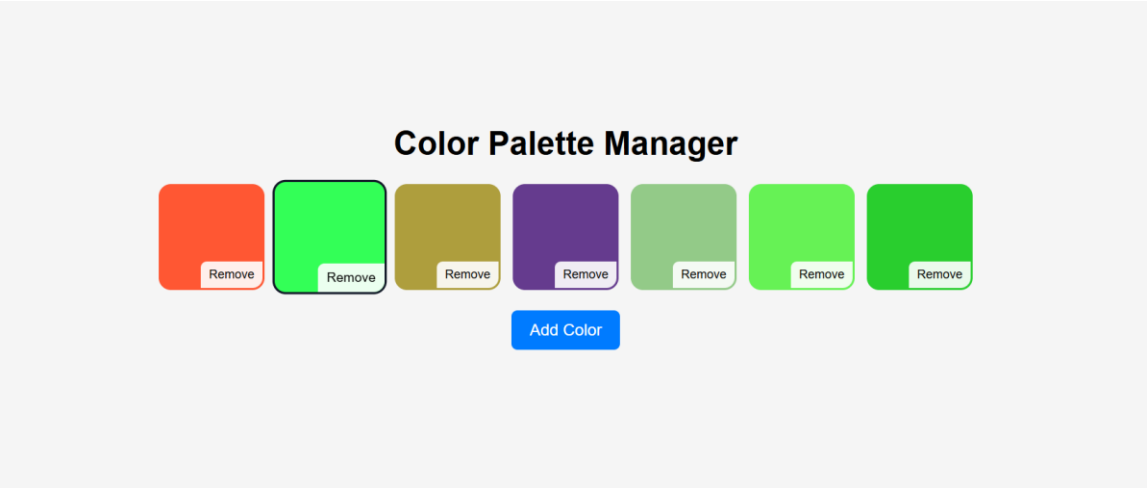
function toggleTodo(id) {
  const items = loadTodos();
  const updated = items.map((item) =>
    item.id === id ? { ...item, completed: !item.completed } : item
  );
  saveTodos(updated);
  renderTodos();
}

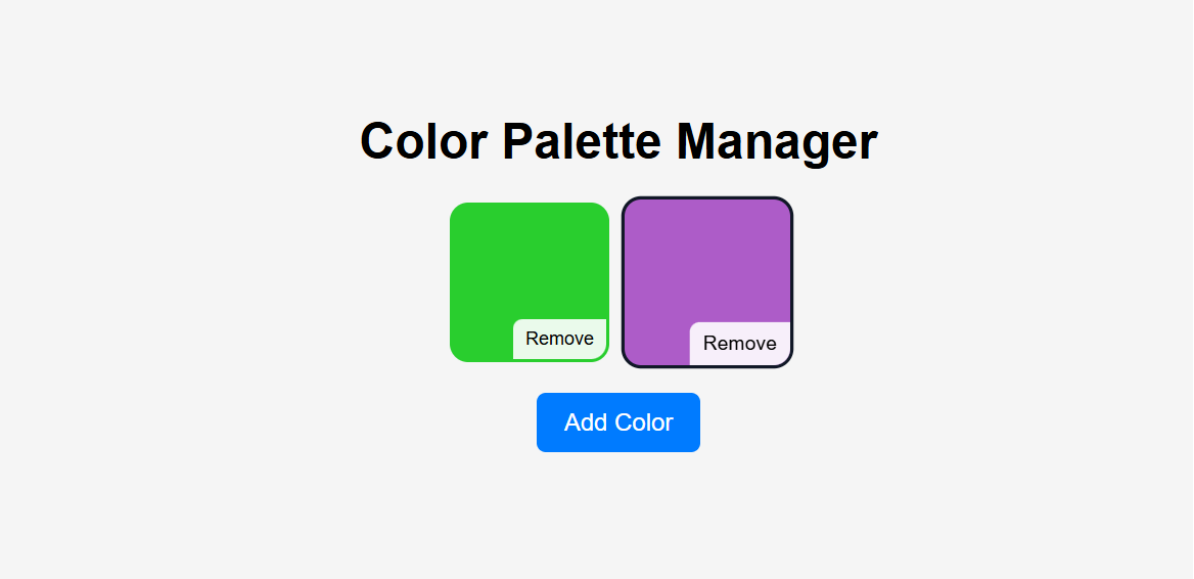
function deleteTodo(id) {
  const items = loadTodos().filter((item) => item.id !== id);
  saveTodos(items);
  renderTodos();
}
```

Q2.

The React app displays selectable color swatches and lets users add a random color or remove a swatch. Context shares the palette and selection state across the components.








```jsx
export const useColorPalette = () => {
  const [colors, setColors] = useState(['#ff5733', '#33ff57', '#5733ff']);
  const [selectedColor, setSelectedColor] = useState(0);

  const addColor = () => {
    setColors((previous) => [...previous, getRandomColor()]);
  };

  const removeColor = (index) => {
    setColors((previous) => {
      if (previous.length <= 1) {
        return previous;
      }

      const nextColors = previous.filter((_, idx) => idx !== index);
      setSelectedColor((currentSelected) => {
        if (currentSelected === null) return null;
        if (currentSelected >= nextColors.length) {
          return nextColors.length - 1;
        }
        return currentSelected;
      });
      return nextColors;
    });
  };

  const selectColor = (index) => {
    setSelectedColor(index);
  };

  const contextValue = useMemo(
    () => ({
      colors,
      selectedColor,
      addColor,
      removeColor,
      selectColor,
    }),
    [colors, selectedColor]
  );

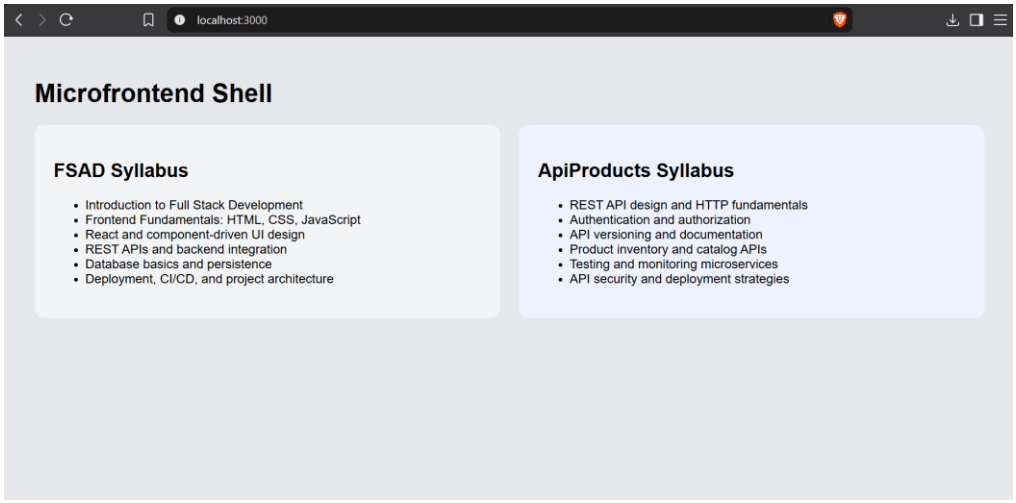
  return contextValue;
};

const ColorPalette = () => {
  const { colors, selectedColor, selectColor, removeColor } = useContext(ColorContext);

  return (
    <div className="color-palette">
```

Q3.

The shell loads the FSAD and ApiProducts syllabus apps as separate Webpack Module Federation remotes. Each remote owns and displays its syllabus, while the shell brings both into one page.



FSAD remote

```jsx
export default function FsadApp() {
  return (
    <section>
      <h2>FSAD Syllabus</h2>
      <ul>
        {syllabus.map((item) => <li key={item}>{item}</li>)}
      </ul>
    </section>
  );
}
```

```jsx
new ModuleFederationPlugin({
  name: 'fsadApp',
  filename: 'remoteEntry.js',
  exposes: {
    './FsadApp': './src/FsadApp',
  },
  shared: {
    react: { singleton: true, requiredVersion: '^18.3.1' },
    'react-dom': { singleton: true, requiredVersion: '^18.3.1' },
  },
})
```

ApiProducts remote

```
new ModuleFederationPlugin({
  name: 'apiProducts',
  filename: 'remoteEntry.js',
  exposes: {
    './ApiProductsApp': './src/ApiProductsApp',
  },
  shared: {
    react: { singleton: true, requiredVersion: '^18.3.1' },
    'react-dom': { singleton: true, requiredVersion: '^18.3.1' },
  },
})
```

Shell

```
new ModuleFederationPlugin({
  name: 'shell',
  remotes: {
    fsadApp: 'fsadApp@http://localhost:3001/remoteEntry.js',
    apiProducts: 'apiProducts@http://localhost:3002/remoteEntry.js',
  },
  shared: {
    react: { singleton: true, requiredVersion: '^18.3.1' },
    'react-dom': { singleton: true, requiredVersion: '^18.3.1' },
  },
})
```

```
const FsadApp = React.lazy(() => import('fsadApp/FsadApp'));
const ApiProductsApp = React.lazy(() => import('apiProducts/ApiProductsApp'));

<Suspense fallback={<div>Loading FSAD module...</div>}>
  <FsadApp />
</Suspense>
<Suspense fallback={<div>Loading ApiProducts module...</div>}>
  <ApiProductsApp />
</Suspense>
```

Q4.

The Go program exposes `add` and `multiply` to JavaScript using `syscall/js` and is compiled to WebAssembly. The Node.js host loads the module, calls both functions, and prints their results.

Output

```
PS C:\Users\venka\Documents\Github\FullStack-assignment> cd Q4
PS C:\Users\venka\Documents\Github\FullStack-assignment\Q4> $env:GOOS = 'js'
>> $env:GOARCH = 'wasm'
PS C:\Users\venka\Documents\Github\FullStack-assignment\Q4> go build -o arithmetic.
wasm main.go
PS C:\Users\venka\Documents\Github\FullStack-assignment\Q4> Remove-Item Env:GOOS
>> Remove-Item Env:GOARCH
PS C:\Users\venka\Documents\Github\FullStack-assignment\Q4> node host.js
add(10, 5) = 15
multiply(10, 5) = 50
PS C:\Users\venka\Documents\Github\FullStack-assignment\Q4>
```

WebAssembly module

```
addFunc = js.FuncOf(func(_ js.Value, args []js.Value) any {
    return args[0].Int() + args[1].Int()
})
multiplyFunc = js.FuncOf(func(_ js.Value, args []js.Value) any {
    return args[0].Int() * args[1].Int()
})

js.Global().Set("add", addFunc)
js.Global().Set("multiply", multiplyFunc)
```

JavaScript host

```
const { instance } = await WebAssembly.instantiate(
  fs.readFileSync(wasmPath),
  go.importObject,
);

const runPromise = go.run(instance);
await ready;

console.log('add(10, 5) =', globalThis.add(10, 5));
console.log('multiply(10, 5) =', globalThis.multiply(10, 5));

globalThis.shutdownWasm();
await runPromise;
```